

Spring Boot

Spring Web RESTful

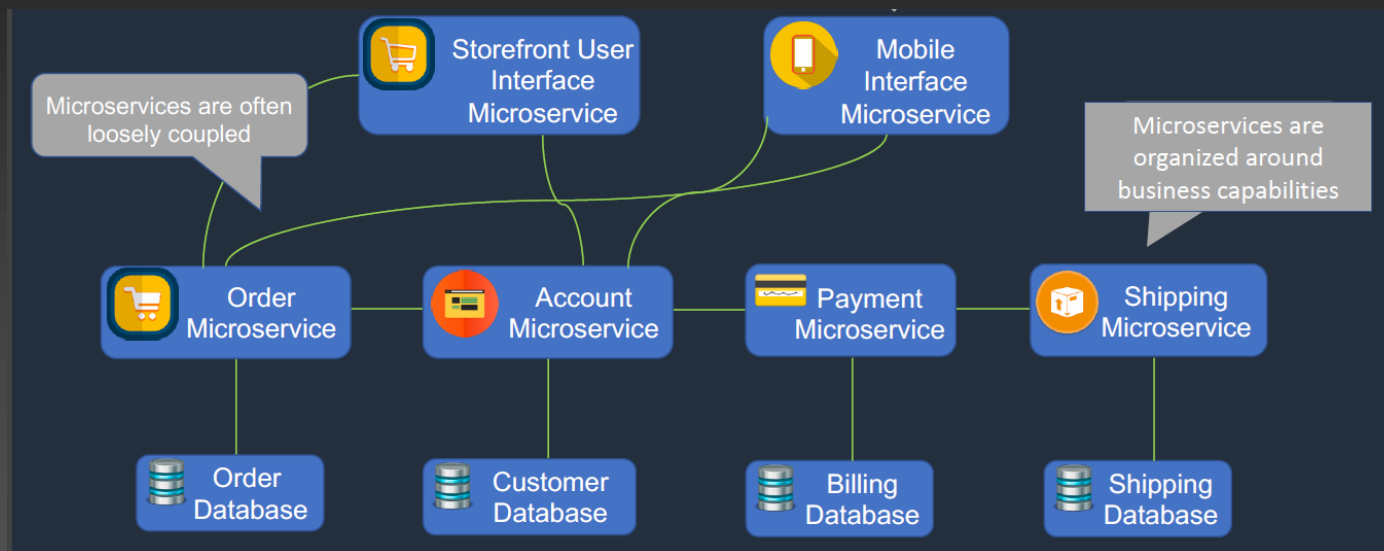
Architettura Monolitica



Architettura Microservizi

- I microservizi sono un approccio per sviluppare e organizzare l'architettura dei software secondo cui quest'ultimi sono composti di servizi indipendenti di piccole dimensioni che comunicano tra loro tramite API ben definite.

Architettura Microservizi



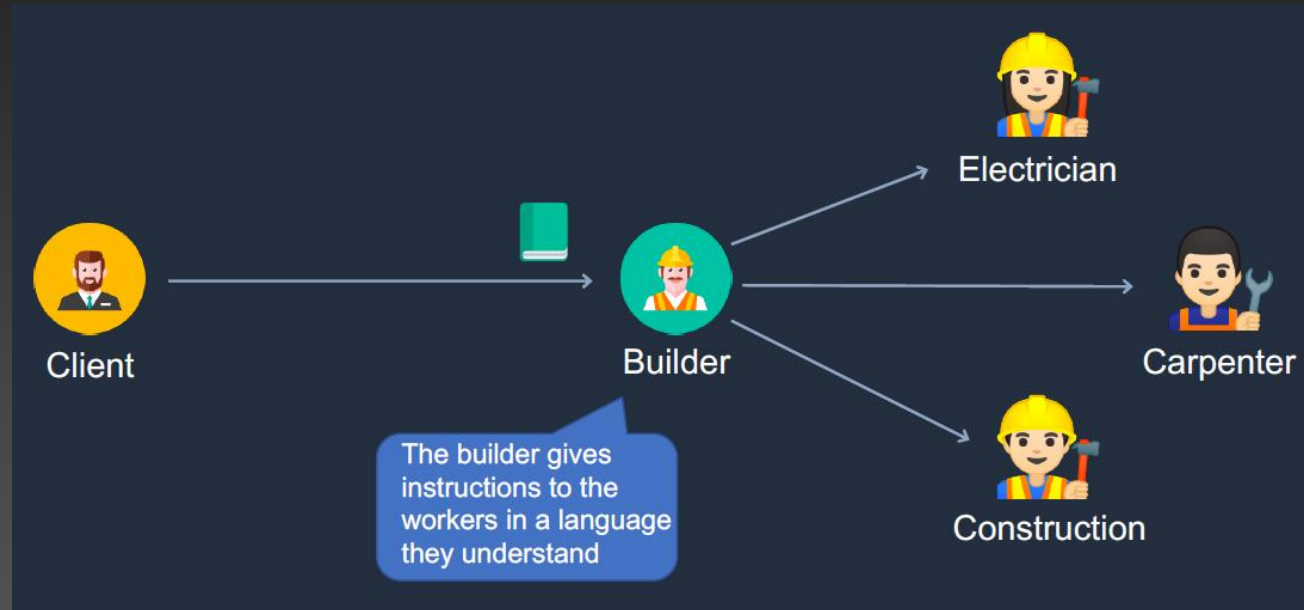
Architettura Microservizi

- Caratteristiche:
 - Autonomi: sviluppato, distribuito, eseguito senza influire gli altri.
 - Specializzati: risoluzione di un problema specifico.
- Vantaggi:
 - Agilità: team indipendenti di sviluppatori con dimensioni ridotte.
 - Scalabilità e flessibilità: scalare servizio in modo indipendente.
 - Semplicità di distribuzione: integrazione e distribuzione continua.
 - Libertà tecnologica: ogni servizio può usare tecnologie diverse.
 - Codice riutilizzabile: utilizzare funzioni per più scopi
 - Resilienza: in caso di errore non si blocca l'intera applicazione.

Cos'è un'API

- API è l'acronimo Application Programming Interface, ovvero un insieme di definizioni e protocolli per la creazione e l'integrazione di software applicativi.
- Meccanismi che consentono a due componenti software di comunicare tra loro:
 - Client - Server
 - Server – Server
- Diversi tipologie di API:
 - RESTFul
 - SOAP
 - RPC
 - Websocket

Cos'è un'API



Cos'è un'API RESTful

- Un'API REST è un'API conforme ai principi di progettazione dello stile architetturale REST.
- REST «REpresentational State Transfer» è uno stile architetturale per sistemi distribuiti.
- I servizi web REST rappresentano un approccio differente ai servizi web rispetto a quello offerto dai servizi basati su SOAP, RPC.
- Usa il protocollo HTTP per eseguire chiamate tra i componenti SW.
- Al giorno d'oggi le architetture a microservizi web REST sono molto comuni.
- Roy Fielding è il creatore:
- http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Proprietà architettura REST

- Client - Server:
- Client e server sono due applicazioni separate.
- In determinati contesti un server ha il ruolo di «client», e un altro server ha il ruolo «server».
- Possono evolvere indipendentemente.
- Al server non interessa con quali tecnologie e come è stato implementato il client. E viceversa.

Proprietà architettura REST

- Stateless:
 - Comunicazione tra client e server è stateless.
 - Il server non memorizza nessuna informazione di sessione che riguarda il client.
 - Tutte le informazioni di cui il server ha bisogno per elaborare una risposta devono essere contenute nella richiesta.
 - Migliora la scalabilità: non esistono comunicazioni interne al server per la gestione della sessione. I componenti sono indipendenti, quindi possono scalare facilmente.
 - Performance di rete peggiori: richieste di grandi dimensioni

Proprietà architettura REST



- Caching lato client:
 - Allo scopo di non fare sempre la stessa richiesta, il client può memorizzare la risposta dal server nella sua cache.
 - Questo è consentito solo se il server applica una configurazione all'header «cache-control» della risposta che vuole far memorizzare nel client.
 - Migliora le performance.
 - Lo svantaggio è che il client potrebbe utilizzare dei dati «obsoleti».



Proprietà architettura REST

- Uniform interface:
 - E' ciò che differenzia le API REST dalle altre.
 - L'interazione con un'API REST è identica per diverse tipologie di applicazioni client(smartphone, browsers, servizi ecc.)
 - Per raggiungere questo obiettivo ci sono quattro vincoli:
 - Identificazione delle risorse: es: «/user/23» è l'URI di una risorsa
 - Manipolazione delle risorse tramite rappresentazioni: JSON ecc..
 - Richieste autodescrittive: ogni richiesta deve contenere il necessario
 - HATEOS(Hypermedia As The Engine Of application State):
 - URI di altre risorse nella risposta da inviare al client. Il client scopre risorse utili(spring-boot-starter-HATEOS)

Proprietà architettura REST

- Architettura a livelli:
 - Architettura composta da diversi livelli. Ogni livello ha la propria responsabilità(es: MVC).
 - Ogni livello non deve conoscere come sono implementati gli altri.
 - Migliora sicurezza: ogni layer è disaccoppiato dagli altri.
 - Peggiora latenza: la richiesta deve passare per tutti i livelli.

Proprietà architettura REST

- Codice on demand(opzionale):
 - Il server può inviare al client delle risposte che contengono codice eseguibile (es: Javascript ecc..)
 - Potenziali problemi di sicurezza.

Risorsa

- Una risorsa è una vera e propria informazione che il server può fornire (es: un prodotto, una collezione di prodotti ecc..)
- Le risorse sono identificate univocamente tramite un URL
 - <http://localhost:8080/myapp/resources/users> --> Collezione di risorse
 - <http://localhost:8080/myapp/resources/users/23> --> Singola risorsa
- I metodi HTTP sono usati per eseguire determinate operazioni sulle risorse: es. leggere, aggiungere, modificare, eliminare una risorsa.

Risorsa



Method	Purpose
GET	Read, possibly cached
POST	Update or create without a know ID
PUT	Update or create with a know ID
DELETE	Remove
HEAD	Read headers, has version changed?
OPTION	List the "Allow"ed methods



Headers HTTP



Esempio di request headers

```
GET /home.html HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0)
Gecko/20100101 Firefox/50.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/testpage.html
Connection: keep-alive
Upgrade-Insecure-Requests: 1
If-Modified-Since: Mon, 18 Jul 2016 02:36:04 GMT
If-None-Match: "c561c68d0ba92bbeb8b0fff2a9199f722e3a621a"
Cache-Control: max-age=0
```

Esempio di response headers

```
200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Mon, 18 Jul 2016 16:06:00 GMT
Etag: "c561c68d0ba92bbeb8b0ff612a9199f722e3a621a"
Keep-Alive: timeout=5, max=997
Last-Modified: Mon, 18 Jul 2016 02:36:04 GMT
Server: Apache
Set-Cookie: mykey=myvalue; expires=Mon, 17-Jul-2017 16:06:00 GMT;
Max-Age=31449600; Path=/; secure
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding
X-Backend-Server: developer2.webapp.scl3.mozilla.com
X-Cache-Info: not cacheable; meta data too large
X-kuma-revision: 1085259
x-frame-options: DENY
```



Rappresentazioni delle Risorse (HTTP Body)



- I componenti REST eseguono azioni su una risorsa usando una rappresentazione per acquisire lo stato corrente o previsto di tale risorsa e trasferendo tale rappresentazione tra i componenti.
- Esempio: per modificare un prodotto(la risorsa) il client deve fornire al server una rappresentazione prevista della risorsa(prodotto modificato).
- Le rappresentazioni supportate sono:
 - JSON
 - XML
 - HTML
 - altri
- HTTP Headers «accept» e «content-type» usati per negoziare il tipo di rappresentazione.



Rappresentazione JSON

- La più comune al giorno d'oggi.
- JSON (JavaScript Object Notation) è un semplice formato per lo scambio di dati tra componenti SW.
- È un insieme di coppie chiave-valore
- Vantaggi:
 - Per le persone facile da leggere e scrivere
 - Per le macchine facile da generare e analizzare
 - Indipendente dal linguaggio di programmazione

Rappresentazione JSON

```
JSON Object → {  
    "company": "mycompany",  
    "companycontacts": {  
        "phone": "123-123-1234",  
        "email": "myemail@domain.com"  
    },  
    "employees": [  
        {  
            "id": 101,  
            "name": "John",  
            "contacts": [  
                "email1@employee1.com",  
                "email2@employee1.com"  
            ]  
        },  
        {  
            "id": 102,  
            "name": "William",  
            "contacts": null  
        }  
    ]  
}
```

String Value

Object Inside Object

JSON Array

Array Inside Array

Number Value

Null Value

Status code

- Lo status code di una risposta HTTP indica al client se una specifica richiesta HTTP è stata completata correttamente.
 - 1xx - risposte Informative:
 - Abbiamo ricevuto la tua richiesta e la stiamo processando
 - 2xx - risposte di successo:
 - Abbiamo ricevuto con successo la richiesta, ecco la risposta.
 - 3xx - risposte di reindirizzamento:
 - Serve un'ulteriore azione per soddisfare la richiesta.
 - 4xx - risposte errore lato client:
 - La richiesta contiene errore di sintassi o non può essere soddisfatta.
 - 5xx - risposte errore lato server:
 - Il server non è riuscito a soddisfare una richiesta apparentemente valida.

Spring Boot REST



- Starter necessario per sviluppare un microservizio REST in Spring Boot

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-webmvc</artifactId>  
</dependency>
```



Spring Boot REST

- Per creare un'API REST dobbiamo creare un classe annotata con `@RestController`.
- L'URI della risorsa è specificato con l'annotazione `@RequestMapping`
- Tale annotazione si può utilizzare sia sulla definizione della classe che sulla definizione del metodo.
- I metodi della classe `@RestController` si chiamano endpoint.

```
@RestController
@RequestMapping(value = "/api/v1")
public class HelloController{

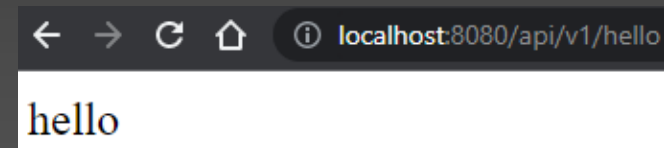
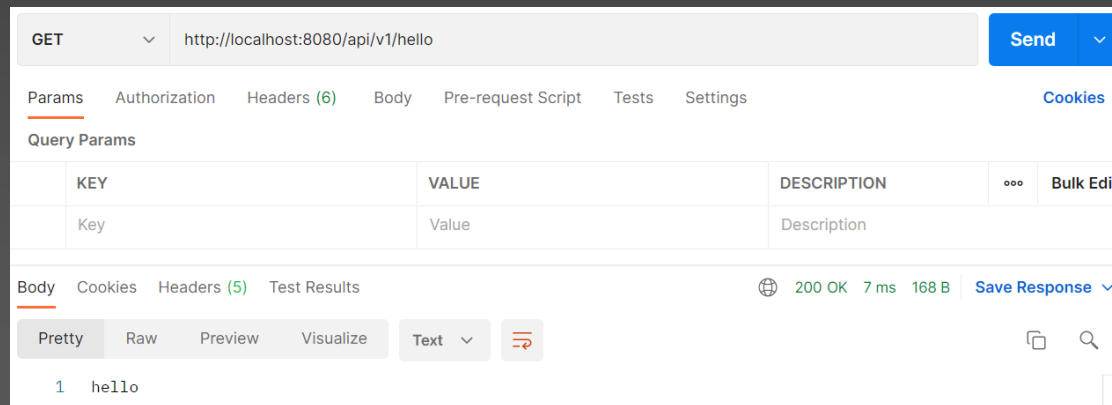
    @RequestMapping(method = RequestMethod.GET, value = "/hello")
    public String hello() {
        return "hello";
    }
}
```

URI: /api/v1/hello
URL: http://host:porta/api/v1/hello

Spring Boot REST



- Per testare le API possiamo usare:
 - Classi di testing di Spring Boot
 - Browser
 - Postman
 - altri



Spring Boot e Spring Cloud

Spring Boot

BUILD ANYTHING

Spring Boot is designed to get you up and running as quickly as possible, with minimal upfront configuration of Spring. Spring Boot takes an opinionated view of building production-ready applications.

Spring Cloud

COORDINATE ANYTHING

Built directly on Spring Boot's innovative approach to enterprise Java, Spring Cloud simplifies distributed, microservice-style architecture by implementing proven patterns to bring resilience, reliability, and coordination to your microservices.

- Spring Boot: Sviluppare i microservizi.
- Spring Cloud: Coordinare i microservizi.